

Guía para escritura de GDDs

¿Qué es un GDD?

Un GDD (*Game Design Document*) es un documento hecho por un desarrollador o equipo de desarrolladores para describir una idea de videojuego. Su objetivo principal es comunicar la visión del proyecto a lo largo de todo el equipo, aterrizar ideas y presentar el proyecto a terceros de manera sencilla y fiable. No confundir GDD como documento con el nombre del grupo estudiantil, que también es GDD.

Hay dos métodos para hacer un GDD:

1. **Monolítico:** Un documento tradicional, hecho en un procesador de texto. Son enciclopédicos, extensos y diseñados para la publicación en internet y la lectura intensiva. Suelen escribirse durante o después del desarrollo, y no es extraño que sean escritos por terceros y no por los propios desarrolladores.
2. **Vivo o dinámico:** Un documento no lineal hecho en una herramienta de colaboración (HackMD, Notion, Collanote, etc.), que se actualiza junto con el desarrollo y lo asiste. Son mas adecuados a largo plazo y se acompañan de vínculos, documentos más cortos y contenidos multimedia. Se hacen al inicio o durante el desarrollo por parte de los desarrolladores y no suelen ser adecuados para publicación, pero si para consulta. **No confundir con la documentación.**

Nosotros usamos ambas formas, y ambas son útiles. Pero para el **checkpoint 1** pedimos un GDD monolítico. El GDD vivo se escribe durante el desarrollo y no sigue ninguna estructura específica, aunque esta guía es igual de adecuada para ese tipo de documentos.

¿Porque pedimos un GDD monolítico?

Un GDD monolítico es el que se escribe en un sólo documento exhaustivo, es extenso, poco atractivo de leer, se desactualiza y no se adapta al proceso creativo de un proyecto. Entonces ¿por qué lo usamos?

El propósito principal de seguir esta estructura es porque es perfecta para **argumentar tu proyecto**. Te obliga a responder preguntas mientras el GDD guarda las respuestas. Tu documento no quedará de 100 páginas, quizás unas 20 o 50 como máximo. Eso es perfectamente legible en la práctica, así que no es un archivo infinito e inútil; es un documento accesible y orientado a la planificación.

En todo caso es recomendable solo hacer un GDD monolítico dentro de un tiempo y extensión razonables, y antes de empezar el proyecto. Si inicias el desarrollo, pasa tu GDD a un formato dinámico que te permita iterarlo y adaptarlo a tus avances.

Estructura para GDDs

Estructura vieja

1. Datos generales

- Resumen/Gancho
- Género
- Audiencia objetivo
- Plataformas
- Herramientas

2. Diseño general

- Experiencia del jugador
- Pilares de diseño
- Puntos de venta únicos (USPs)

3. Diseño detallado

- Mecánicas
- Estilo artístico
- Escenario/Mundo
- Historia
- Interfaz
- Música/sonido
- Game loop
- Progresión
- Sistemas

4. Referencias

5. Anexos

Estructura nueva

1. Resumen

- Sinopsis
- Género
- Público objetivo

2. Desarrollo

- Herramientas a usar
- Plataformas objetivo
- Flujo de trabajo

3. Diseño

- Pilares de diseño
- Puntos de venta únicos (USPs)
- Experiencia del jugador
- Game loop
- Progresión
- Sistemas

4. Contenido

- Mecánicas
- Estilo artístico
- Música y sonido
- Mundo y escenarios
- Historia
- Interfaz

5. Referencias

- Inspiraciones
- Contenido preliminar
- Anexos

¿Y lo que falta?

- Mecánicas principales vs secundarias: va en **Mecánicas**
- Game feel: va en **Experiencia del jugador**
- Curva de dificultad y ritmo del juego: va en **Progresión**
- Recompensas y economía: va en **Sistemas**
- Estilo narrativo: va en **Mecánicas** o como explicación a lo largo del documento
- Personajes: van en la **Historia**
- Tutoriales o aprendizaje del jugador: va en **Experiencia del jugador**
- Flujo del usuario: va en **Interfaz** o se intuye del resto del documento
- Roadmap de desarrollo para el semestre: **no va en un GDD**
- Requerimientos técnicos para el juego: **difícil de dimensionar y poco relevante**
- Investigación de mercado: va como justificación/análisis en **Puntos de venta únicos**

Veremos más adelante que cosas sí van en el GDD, además de esto.

Contenido de un GDD

La estructura que recomendamos consiste en 5 secciones: resumen, desarrollo, diseño, contenido y referencias. Cada una busca resolver un frente diferente y generar una visión completa sobre tu proyecto. Es importante destacar que un GDD **no es documentación**, por lo tanto, los detalles finos y de implementación no van en el documento. Un GDD es para características esenciales del proyecto.

1. Resumen

Sinopsis

Un resumen en 1 o 2 frases que explique porque tu juego es divertido o interesante. Como si lo fueras a poner en la descripción para Steam o explicarlo a un amigo, por ejemplo.

Ejemplo Expedition 33 en Steam

«Guía a la expedición 33 en su misión de acabar con la Pintora para que no vuelva a pintar la muerte. Explora un mundo inspirado en la Francia de la Belle Époque y lucha contra rivales únicos en este juego de rol por turnos con mecánicas en tiempo real.»

Género

¿Cuál es el género de tu videojuego? esto suele decidir directamente una gran cantidad de características y lo enmarca en un tipo de juego específico. En [Wikipedia \(inglés\)](#) encuentras una lista para que te guíes. Un juego puede tener más de un género a la vez.

Ejemplo Grand Theft Auto V

GTA V (y toda la saga *Grand Theft Auto*) es un juego de acción, aventura y mundo abierto. Es común que la acción y aventura vayan juntos, de hecho.

Público objetivo

Tu juego **jamás** será para todos. Definir que tipo de persona quieres que juegue tu juego te ayudará a comprender que necesitas hacer. Describe motivaciones, gustos, edades, géneros, e incluso inspiraciones y características que puedan guiar a las personas indicadas a jugar tu juego.

Tu público objetivo no es estricto: técnicamente cualquiera puede jugar tu juego, pero no a todos les va a gustar. Esto último está fuera de tu público objetivo. Además, el público objetivo define claramente quienes no deberían jugar. Un juego +18 no debe ser jugado por menores de edad, mientras que un juego para preescolares es completamente inadecuado o irrelevante para adultos.

Ejemplo Minecraft

Minecraft es uno de los videojuegos más universales de los tiempos modernos. Pero no le impide limitar su público objetivo a:

- Mayores de 8 años (especialmente entre 12 y 24 años)
- Personas que destacan por ser creativas y resilientes
- Educadores y estudiantes

2. Desarrollo

Herramientas a usar

¿Qué herramientas, apps, o software será utilizado para producir el juego? Estos abarcan:

- Motor de videojuegos y lenguajes de programación
- IDE/Editor de texto y código
- Gestión de código y recursos
- Modelado 3D
- Ilustración
- Herramientas de productividad
- Editores de audio
- Librerías o software propio, si es el caso

Existen algunas obviedades que puedes omitir para esta sección:

- Git para control de versiones. Es menos obvio usar otra cosa, eso sí
- WhatsApp y Discord para comunicaciones. Es el canal que todos usamos

Ejemplo Slay the Spire 2

Slay the Spire 2 fue hecho en Unity + C# y luego migrado a Godot + C#. También se usaron librerías propietarias de Mega Crit. No parece haber mejor información sobre el resto de herramientas usadas, y de hecho puedes elegir las que quieras.

Plataformas objetivo

¿En qué plataformas y dispositivos estará disponible el juego? Cada plataforma es un mercado, un público objetivo, unas necesidades a resolver y un estilo que seguir, todas completamente diferentes, por lo que influye mucho en lo que tu juego puede ser y tener. Por ejemplo un juego competitivo o de paga seguramente no es adecuado para teléfonos, mientras que uno hipercasual no es muy adecuado para consolas. Cada plataforma es un paradigma de juego diferente.

Ejemplo Undertale

Undertale fue lanzado oficialmente para:

- Windows
- macOS
- Linux (SteamOS)
- Nintendo Switch y Switch 2
- Steam Deck
- Xbox One y Series X y S
- Playstation 4 y Vita

Flujo de trabajo

¿Cuál es la forma en la que van a trabajar como equipo/individuo? ¿Cuáles son sus prioridades? ¿Pruebas rápidas, feedback, repartición de roles? Definir esto les puede despejar dudas para evitar atorarse como grupo.

Ejemplo Deltarune

Deltarune es desarrollado principalmente sobre el prototipado e ideas de Toby Fox, refinado por Temmie Chang e implementado/asistido por Toby más otros desarrolladores, algunos de 8-4 para la versión japonesa y de consolas.

Por lo que conocemos, parece involucrar un trabajo fino y directo hacia el contenido final. Sin iteración masiva, o feedback de su comunidad.

3. Diseño

Pilares de diseño

¿Qué rige el desarrollo de tu juego? Como si fuesen mandamientos para el grupo, los pilares de diseño son principios simples y claros que deben dirigir tu desarrollo. A diferencia de el contenido, los pilares de diseño suelen ser no cambiar durante todo el desarrollo, y plasman las ambiciones, filosofía y necesidades de tu equipo y el proyecto.

Los pilares de diseño son útiles para contrastar y dirigir decisiones, evitando perder el rumbo o arruinar el proyecto con ideas incompatibles. Definirlos al principio también aterriza los objetivos del grupo y la forma en la que trabajarán.

Los pilares de diseño pueden ser de dos tipos: prácticos, para decidir como debe ser la jugabilidad y el contenido; y filosóficos, para decidir como el equipo debe pensar sobre tu proyecto. Se recomiendan entre 3 y 5 principios bien explicados, de cualquier tipo o combinación de ellos.

Ejemplo Hades

Hades, de Supergiant Games, se guía por la fusión entre los roguelikes implacables con la narrativa progresiva y dinámica. Sus principios son principalmente prácticos:

- Usar el fracaso como motor de la narrativa
- Minimizar el contenido repetitivo
- Combate rápido, frenético y súper dinámico
- Aspecto eléctrico y dinámico
- Música adaptativa e inmersiva

Puntos de venta únicos (USPs)

Un punto de venta único (*Unique Selling Points*, USP) es un elemento de tu juego que lo hace único. Plantear estos puntos para tu juego no es obligatorio, pero puede ser esencial para destacar y darle personalidad a tu proyecto.

Antes de plasmarlos, evita hacer plagio y consulta el mercado actual en búsqueda de novedades. También es relevante que los USPs tengan algo de tu estilo o personalidad, así puedes interiorizar mejor tu idea y facilitar el ejercicio de la creatividad.

Ejemplo Índigo Park: Chapter 1

Índigo Park es que es un juego de *horror de mascotas* (como FNAF) creado por Mason Myers (*UniqueGeese*). Uno de sus USPs es que tienes a una mascota de tu lado que guía tu experiencia, Rambley; además, tiene una historia relevante para el juego y te la cuenta él con una ternura contrastante. Esto subvierte su género, que suele tener a todos los personajes como enemigos, sin contexto y sin mayor atractivo que los *screamers* o un lore cuestionable.

Experiencia del jugador

¿Qué experiencia se espera generar en el jugador durante todo el juego? ¿Que fantasía queremos alimentar en el jugador? Usualmente, la experiencia deseada se puede explicar como:

- Transmitir una sensación o emoción
- Ser/personificar a un personaje
- Hacer una actividad en particular

Ejemplo Undertale

Undertale es bien conocido por subvertir los RPGs al introducir el concepto de perdón y poder de decisión a lo largo de la historia. La experiencia aboga por sentir valor en tus acciones y actuar de corazón más que por «piloto automático para jugar RPGs».

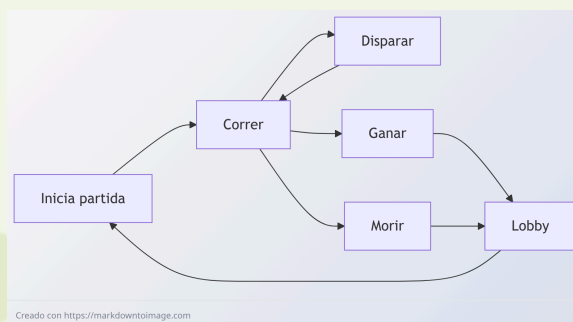
Game loop

Un *game loop* es un diagrama en bucle o explicación que describe el núcleo de las mecánicas del jugador, y el orden y posibilidades en las que el jugador hace sus acciones. Esto se considera diseño, y no contenido, debido a que el game loop no depende del contenido, y en cambio, lo dirige.

El game loop está estrechamente relacionada con el género de tu videojuego, así que puedes empezar consultando el game loop típico de los géneros que abarca tu proyecto, y luego adaptándolo a tu gusto o necesidad. Puedes hacer los diagramas en Mermaid, Miro, Canva o la herramienta de tu gusto. O explicarlo en texto si es más adecuado.

Ejemplo

Los juegos de disparos/FPS (como Fortnite) suelen compartir el mismo bucle básico:



Otras acciones como áreas seguras o loot dependen del juego.

Progresión

¿Cómo progresa el jugador en tu juego? ¿Cuáles son sus objetivos a corto y largo plazo? ¿Están estos objetivos alineados con la fantasía que se quiere transmitir? Igual que el game loop, esto se suele definir con base en el género que elegiste.

Por supuesto, la progresión depende también del contenido y la historia de tu juego, aunque no lo define *per se*.

Ejemplo

Saga Pokémon

En Pokémon, tus objetivos a corto plazo son conseguir medallas y mejorar a tus Pokémon, todo con el fin de acercarte a derrotar al Alto mando y luego, al campeón de la región.

Sistemas

¿Qué sistemas posee tu juego? ¿Como impactan o dirigen la experiencia del jugador? Los sistemas suelen abarcar:

- Economía y manejo de recursos
- Rangos, escalas y puntajes
- **Funcionamiento** de mecánicas clave. El *contenido* y contexto de esas mecánicas va más adelante

No debes incluir implementaciones ni detalles finos de los sistemas (ej: fórmulas del sistema Elo o precios de artículos clave), pues no son relevantes. Esos van en la documentación del proyecto, y acá solo explicas como funcionan y porque son importantes.

Algunos ejemplos triviales que puedes omitir:

- Interfaz de usuario (asumimos que todos los juegos tienen esto)
- Sistemas de la implementación (ej: Sistema de programación con *Blueprints*)
- Sistema de movimiento para personajes (un poco obvio)

Puedes incluirlos si son lo suficientemente relevantes o únicos para la identidad de tu juego, pero si son sistemas comunes o esenciales a cualquier juego, mejor omitirlos.

Ejemplo Ajedrez

El ajedrez tiene su característico sistema Elo, pero no es muy adecuado explicar la fórmula, sino los conceptos claves del sistema y como impacta a los jugadores: en este caso, el Elo se usa para armar partidas y clasificar jugadores.

Por otro lado, explicar las reglas del ajedrez como un sistema sería irrelevante, aunque para un videojuego (y uno nuevo) quizá no es el caso.

4. Contenido

Mecánicas

¿Qué puede hacer el jugador en el juego? Describe brevemente las mecánicas principales que componen el juego, normalmente en términos de verbos/acciones. Las mecánicas son el corazón de tu juego, aunque no sean algo innovador en sí mismo, quizás la combinación de mecánicas o la idea detrás de ellas sí.

Ejemplo Hollow Knight

En Hollow Knight puedes explorar el mundo, adquirir habilidades, mejorar las herramientas y habilidades de tu personaje y pelear contra enemigos y jefes.

Estilo artístico

El estilo artístico de tu juego es la forma en la que se ve. Puede ser arte vectorial, *low poly*, *pixel art*, ilustrado, realista, entre otros. Es recomendable atender a estilos específicos y bien descritos, por ejemplo, no decir *pixel art*, sino explicar que tipo de *pixel art* (8-bit, Hi-res, rotoscopia-3D).

Ejemplo Deltarune

Deltarune usa *pixel art*, pero no basta decir eso. Sigue una técnica tradicional, resolución baja-media y un estilo dinámico y moderno, sin dejar de lado sus raíces retro.

Si lo comparas con Undertale, del mismo autor, notarás que sus estilos ya son significativamente diferentes. Por eso es importante hacer especificación sobre que tipo de arte queremos.

Música y sonido

¿Qué estilo musical se utilizará en el juego? ¿Cómo influye la música en otros aspectos del juego? ¿Cómo contribuye la música y el sonido a la experiencia del jugador?

Ejemplo Devil May Cry

La música de Devil May Cry es activa y emocionante, generando en el jugador un sentimiento de “adrenalina” que lo incentiva a querer luchar con los enemigos y destruirlos utilizando combos.

Mundo y escenarios

¿En qué mundo se desarrolla el juego? ¿Cuáles son las características de este mundo? ¿Cuáles son los acontecimientos principales que distinguen el mundo en que se presenta el juego? Además de la construcción del mundo, también puede ser adecuado añadir transfondo o *lore*, aunque también puede ir en la historia más adelante.

Evita incluir detalles demasiado específicos o irrelevantes.

Ejemplo Devil May Cry 3

En Devil May Cry 3, estás en un universo donde existen los demonios y los humanos. En este mundo, el antiguo demonio Sparda selló la puerta que está entre el mundo humano y el de los demonios. En el juego, uno de los dos hijos que tuvo Sparda con una humana, Vergil, en busca de más poder, trata de abrir de nuevo la puerta al mundo demoníaco y su hermano gemelo, Dante, decide detenerlo.

Historia

¿Cuáles son los puntos principales de la historia que se va a contar en el juego?

Ejemplo God of War

En God of War se cuenta la historia de Kratos, un guerrero espartano que le vendió su alma al dios de la guerra, Ares, en un enfrentamiento contra otro ejército. Sin embargo, al vender su alma, Kratos acaba asesinando a su familia por una trampa de Ares, convirtiéndose en el Fantasma de Sparta. Kratos decide seguir órdenes de los dioses para que los mismos le borren los recuerdos de su familia.

En el juego, Kratos se embarca en una misión para obtener la caja de Pandora con el fin de derrotar a Ares a petición de los dioses. Luego de algunos giros de trama, Kratos acaba enfrentándose a Ares, y al matarlo, se convierte en el nuevo dios de la guerra.

Interfaz

¿Cómo interactúa el jugador con el juego? ¿Cómo el juego comunica información importante al jugador? Ten en cuenta controles, entradas, menús y elementos de la interfaz gráfica en general.

Ejemplo Doki Doki Literature Club & Minecraft

En las novelas visuales, como Doki Doki Literature Club, todo se basa en tomar algunas decisiones, realizar algunos minijuegos y dar click para continuar con los diálogos. El jugador interactúa con estas opciones de continuar un diálogo y tomar una decisión.

En Minecraft, tienes un menú de inventario, y puedes interactuar rompiendo bloques, poniéndolos, construyendo herramientas, etc. Además, en este juego tienes un indicador de vida y de salud que te indica si estás a punto de morir o si necesitas comer para no morir de hambre.

5. Referencias

Inspiraciones

¿Cuáles obras te inspiran a hacer tu juego? Es común y casi inevitable que sean otros juegos, pero también pueden ser libros, películas, experiencias y mucho más. Explica en que te inspiras y que quieres retomar de esos proyectos. Para proyectos no muy conocidos quizá sea adecuado un link o explicación.

Ejemplo

Beast Card Clash

Beast Card Clash es uno de los proyectos más avanzados y recientes de GDD. Está inspirado fuertemente en Jiu-jitsu Fuego de Club Penguin, y recuerda a elementos de mundo abierto y roguelike sencillos. La estética cozy que busca es familiar a juegos como *Animal Crossing*.

Contenido preliminar

Aquí puedes añadir cualquier cosa que ya tengas para tu proyecto: bocetos, ideas rápidas de contenido, evidencias de un prototipo, etc.

Anexos (opcional)

Cualquier nota o elemento que quieras añadir y que no sea de ninguna categoría del documento va acá.



Bibliografía

Recursos

- Estructura profesional de un GDD por UDIT
- Documento de diseño de videojuegos - Wikipedia
- Tutorial para GDDs por Alva Mayo
- Ejemplos de GDD (en inglés)

Juegos mencionados

- Expedition 33 (2025)
- Saga GTA (desde 1997)
- Minecraft (2011)
- Slay the Spire 2 (2026)
- Undertale (2015)
- Deltarune (desde 2018)
- Hades (2020)
- Índigo Park: Chapter 1 (desde 2024)
- Fornite (2017)
- Saga Pokémon (desde 1996)
- Ajedrez
- Hollow Knight (2017)
- Devil May Cry (2001)
- Devil May Cry 3 (2005)
- God of War (2004)
- Doki Doki Literature Club (2017)
- Beast Card Clash (desde 2025, en desarrollo)

